AI-DRIVEN SOFTWARE ENGINEERING · FALL 2026 · KBTU SITE

Practice #08 — Fix and Improve a Buggy Codebase

1 point · branch and folder `week-08` · AI use **Level D** · due **Sunday 8 November, 23:59** · needs your Practice #07 suite.

We broke canBook on purpose — twice. You reproduce each bug with a failing test before touching anything, write a debugging skill by hand, compare its diagnosis with a plain prompt on the same input, and fix each bug with one minimal change.

Your choice: the assistant — ChatGPT or Claude accept uploaded skills on some plans; Claude Code reads a folder. **Cannot install a skill?** Run the deck's prompt-only simulation and declare it — it costs nothing.

The workflow (deck)

```
REPRODUCE → LOCALIZE → EXPLAIN → PATCH → VERIFY → IMPROVE
```

BUG-A — the overlap logic treats touching endpoints as conflicts. **BUG-B** — the function changes the original bookings array. **For each bug, show a failing test before fixing it.**

The steps

1. **Setup.** Branch `week-08`; `mkdir week-08 && git checkout week-07 -- week-07/app && git mv week-07/app week-08/app`; unzip; run your suite (green) → §2.1.
2. **Reproduce.** Copy the whole `canBook` from `buggy/booking.ts` (or `.js`) over yours, change nothing, run → §2.2. Your own hypothesis per failure → §3. **One bug can hide another** — ask why each test failed.
3. **Regression tests.** `BUG-A …`: a touching request, `.toBe(true)`. `BUG-B …`: an Available request that touches nothing (e.g. 720–780), then bookings compared with a copy. Both fail on the buggy code → §2.3.
4. **The skill (by hand).** Type `skills/bug-detector/SKILL.md` below; save as All files (not `SKILL.md.txt`); zip the folder → `bug-detector.zip`; install; screenshot → `evidence/install.png`.
5. **Without the skill.** Fresh chat: "Find the root cause of the failed room-booking test. Propose minimal patch and regression test. Do not edit code." + AC1–AC6, the failing output, the code → §5.
6. **With the skill.** Fresh chat, skill on: "Use the bug-detector skill to diagnose the failed room-booking test." + **exactly the same input** → §6. Compare in §7 against the code, not each other.
7. **Repair.** Review the patch yourself. One minimal change per bug, one commit each; run the failing test, then the full suite → §8.

SKILL.MD — TYPE IT, EXACTLY

```
---
name: bug-detector
description: Diagnose bugs in existing code using acceptance criteria and failing tests. Use when
reviewing the Smart Campus booking application.
---
# Bug Detector

## Procedure
1. Read acceptance criteria, code, and observed failure.
2. Reproduce with the smallest deterministic input.
3. Identify the likely root cause and relevant lines.
4. Separate confirmed defects from hypotheses.
5. Suggest a minimal fix, but do not edit code yet.
6. Propose a regression test and expected result.
7. Output: defect | evidence | root cause | fix | test.
```

What the checker reads

- Your `canBook` body differs from the provided one in **1 to 4 lines** — a rewritten function is not a minimal change, even if it passes.
- Touching is no longer a conflict; nothing in `canBook` changes `bookings`; T01–T14 still there; `BUG-A` and `BUG-B` tests present.
- The ZIP holds `bug-detector/SKILL.md`, identical to the committed file; your two chat inputs are at least 90% alike.

COMMANDS

```
cd week-08 && python tests/check debugging.py      (31 checks)
              python tests/validate submission.py
```

Submit and grading

PR `week-08 → main`, left open; the PR link in the Teams assignment **Practice #08**. Not an e-mail.

Criterion (0.25 each)	Complete when …
Reproduction and repair	Both bugs fail a test before any fix; your own hypotheses; one minimal change per bug, one commit each; final suite green with BUG-A and BUG-B.
The skill	SKILL.md typed, valid, identical in the ZIP; installed and evidenced, or the simulation declared.
Comparison and judgement	Same input in both chats; both questions as written; every row of §7 judged against the code; a real limitation.
Evidence and honesty	§9, §10 your own; AI_USAGE.md; submission.yml validating and matching the re-run; open PR, 3+ commits.

A FAIL you report and explain costs you nothing. One you hide costs the whole criterion.